Global Warming Science

https://courses.seas.harvard.edu/climate/eli/Courses/EPS101/

Clouds!

Please use the template below to answer the workshop questions. "XX" indicates places where you need to complete/write code or add a discussion.

your name:

[ ]:
```
# import libraries and get data:
import sys
import numpy as np
from matplotlib import pyplot as plt
from scipy import optimize
import pickle
from scipy.integrate import solve_ivp
# cortopy allows to plot data over maps in various spherical prjections"
import cartopy.crs as ccrs
import cartopy.feature as cfeature
from cartopy.feature import NaturalEarthFeature
from cartopy import config
from mpl_toolkits.axes_grid1 import make_axes_locatable

# load the data from a pickle file:
with open('./clouds_variables.pickle', 'rb') as file:
    while 1:
        try:
            d = pickle.load(file)
        except (EOFError):
            break
            # print information about each extracted variable:
        for key in list(d.keys()):
            print("extracting pickled variable: name=", key,  "; size=", d[key].shape)
            #print("type=",type(d[key]))
        globals().update(d)
```

Explanation of input data:

The input variables are results from two prominent climate models (GFDL, Geophysical Fluid Dynamics Laboratory in Princeton NJ, and the Hadley center in the UK. The variable Delta_LW_CRF_GFDL, for example, then represents

$$\Delta CRF_{LW} = CRF_{RCP8.5,LW} - CRF_{historical,LW}$$

in W/m^2 and similarly for the other cloud variables:

Delta_LW_CRF_GFDL, Delta_SW_CRF_GFDL,

Delta_LW_CRF_Hadley, Delta_SW_CRF_Hadley

The variables Delta_SAT_GFDL/Hadley are the difference in surface air temperature (K) between RFP8.5 at 2100 and historical run at 1850:

Delta_SAT_GFDL, Delta_SAT_Hadley

The variables clouds_... are maps of the fraction of the sky covered with clouds as function of longitude and latitude and the corresponding lat/lon axes. These are given for "historical" which is pre-wrming and for the RCP8.5 projection:

clouds_GFDL_historical, clouds_GFDL_rcp85,

clouds_Hadley_historical, clouds_Hadley_rcp85

The lat/lon axes for all of the above variables are given by:

model_clouds_latitude, model_clouds_longitude

1) Clouds radiative forcing and climate sensitivity in climate models: Plot the results of two prominent climate models (from the Geophysical Fluid Dynamics Laboratory, Princeton, NJ, and the Hadley Center, UK), and their difference. Discuss the implications to global warming uncertainty. First, clouds (three contour plots per each of the two models):

(a) The global distribution of cloud fraction at year 1850.

(b) The projected global distribution of cloud fraction at year 2100, according to the RCP8.5 scenario.

(c) The difference between the two.

[ ]:
```
# plot clouds in two models:
# --------------------------

lon = model_clouds_longitude
lat = model_clouds_latitude

# preliminaries, defining configuration of subplots:
projection=ccrs.PlateCarree(central_longitude=0.0)
fig,axes=plt.subplots(3,2,figsize=(18,9) \
            ,subplot_kw={'projection': projection},dpi=300)


# GFDL historical:
# ----------------
mylevels=np.arange(XX,XX,XX);
axes[0,0].set_extent([-180, 180, -90, 90], crs=ccrs.PlateCarree())
axes[0,0].coastlines(resolution='110m')
axes[0,0].gridlines()
c=axes[0,0].contourf(lon,lat, clouds_GFDL_historical,levels=mylevels)
clb=plt.colorbar(c, shrink=0.95, pad=0.02,ax=axes[0,0])
clb.set_label('Cloud fraction %')
axes[0,0].set_title('clouds GFDL historical')


# GFDL rcp8.5:
# ------------
axes[1,0].set_extent([-180, 180, -90, 90], crs=ccrs.PlateCarree())
axes[1,0].coastlines(resolution='110m')
axes[1,0].gridlines()
#c=axes[1,0].contourf(XX)
# XX
```

```
axes[1,0].set_title('clouds GFDL rcp8.5')


# GFDL difference:
# ----------------
# XX
# use colormap cmap='bwr' to plot the difference between two fields
clb.set_label('Cloud fraction %')
axes[2,0].set_title('clouds GFDL rcp8.5$-$historical')

# Hadley historical:
# ------------------
axes[0,1].set_extent([-180, 180, -90, 90], crs=ccrs.PlateCarree())
axes[0,1].coastlines(resolution='110m')
axes[0,1].gridlines()
c=axes[0,1].contourf(lon,lat, clouds_Hadley_historical)
clb=plt.colorbar(c, shrink=0.95, pad=0.02,ax=axes[0,1])
clb.set_label('Cloud fraction %')
axes[0,1].set_title('clouds Hadley historical')


# Hadley rcp8.5:
# --------------
# XX
clb.set_label('Cloud fraction %')
axes[1,1].set_title('clouds Hadley rcp8.5')


# Hadley difference:
# ------------------
axes[2,1].set_extent([-180, 180, -90, 90], crs=ccrs.PlateCarree())
# XX
clb.set_label('Cloud fraction %')
axes[2,1].set_title('clouds Hadley rcp8.5$-$historical')


# finalize and show plot:
plt.subplots_adjust(top=0.92, bottom=0.08, left=0.01, right=0.95 \
                    , hspace=0.15,wspace=-0.4)
plt.show()
```

Next, temperature and CRE (three contour plots for each of the two models):

(d) The global distribution of the change in surface air temperature (SAT) between the preindustrial state (often referred to as historical and representing the year 1850) and the RCP8.5 scenario at 2100: SAT = SATRCP8.5 − SAT historical.

(e) Change in LW CRE: CRELW = CRELW,RCP8.5 − CRELW,historical.

(f) Change in SW CRE.

[ ]:
```
# plot CRF in two models:
# -----------------------
```

```
# preliminaries, defining configuration of subplots:
projection=ccrs.PlateCarree(central_longitude=0.0)
fig,axes=plt.subplots(3,2,figsize=(18,9),subplot_kw={'projection': projection} \
                      ,dpi=300)
contour_levels_SAT=np.arange(-20,21,1)
contour_levels_CRF=np.arange(-40,41,1)

# GFDL Delta SAT:
# ----------------
axes[0,0].set_extent([-180, 180, -90, 90], crs=ccrs.PlateCarree())
#XX
axes[0,0].set_title('GFDL $\Delta$ SAT')


# GFDL Delta CRF SW:
# -------------------
#XX
clb.set_label('CRF (W/M$^2$)')
axes[1,0].set_title('GFDL $\Delta$ CRF$_{SW}$')


# GFDL Delta CRF LW:
# -------------------
contour_levels_diff=np.arange(-20,20.6,0.5)
#XX
clb.set_label('CRF (W/M$^2$)')
axes[2,0].set_title('GFDL $\Delta$ CRF$_{LW}$')

# Hadley Delta SAT:
# ----------------
#XX
axes[0,1].set_title('Hadley $\Delta$ SAT')


# Hadley Delta CRF SW:
# --------------------
#XX
clb.set_label('CRF (W/M$^2$)')
axes[1,1].set_title('Hadley $\Delta$ CRF$_{SW}$')


# Hadley Delta CRF LW:
# --------------------
#XX
clb.set_label('CRF (W/M$^2$)')
axes[2,1].set_title('Hadley $\Delta$ CRF$_{LW}$')


# finalize and show plot:
plt.subplots_adjust(top=0.92, bottom=0.08, left=0.01, right=0.95 \
                    , hspace=0.15,wspace=-0.4)
plt.show()
```

(g) Inter-model difference of the change in SW and LW CRE: Hadley minus GFDL. That is, $\Delta CRE_{LW}^{\text{model 1}} - \Delta CRE_{LW}^{\text{model 2}}$ and the same for the SW CRE.

[ ]:
```
delta_lw_diff = Delta_LW_CRF_Hadley - Delta_LW_CRF_GFDL
delta_sw_diff = XX

fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10, 3),
                       subplot_kw={'projection': ccrs.PlateCarree()},
                       constrained_layout=True
                       )

# Delta LW:
ax[0].add_feature(cfeature.COASTLINE, linewidth=0.8)
ax[0].add_feature(cfeature.BORDERS, linestyle=':', linewidth=0.5)
mesh = ax[0].pcolormesh(
    lon, lat, delta_lw_diff,
    transform=ccrs.PlateCarree(),
    cmap="RdBu_r", vmin=-20, vmax=20,
    shading='auto'
)
ax[0].set_title("$\\Delta$LW CRF: Hadley - GFDL", fontsize=12, pad=10)
plt.colorbar(mesh, ax=ax[0], orientation='vertical', shrink=0.7,
             pad=0.05, label="W/m$^2$")

# Delta SW:
XX
```

Discuss:

XX

2) Convection and cloud formation: Consider an atmospheric vertical temperature profile with a prescribed lapse rate of 6.5 K/km and an air parcel starting at the surface with a temperature equal to that of the atmospheric profile there and at 70% saturation.

First, some preliminaries:

[ ]:
```
# PHYSICAL CONSTANTS
PZERO   = 101325.0 # sea-level pressure, N/sq.m
g       = 9.81     # gravity
R_water = 461.52   # gas constant J/kg/K
R_dry   = 287      # gas constant J/kg/K
cp_dry  = 1005     # J/kg/K
L       = 24.93e5  # latent heat J/kg (vaporization at t.p.)
print("pressure scale height for Tbar=260 K is",R_dry*260/g)

def SimpleAtmosphere(z):
    """ Compute temperature as a function of height in a simplified standard␣
 ↪atmosphere.
    Correct to 20 km. Approximate thereafter.
    Input: z, geometric altitude, m.
    Output: std. temperature
    """
    TZERO   = 298.15   # sea-level temperature, Kelvin
```

```
    if z<11000.0:      # troposphere
        T_a=(298.15-6.5*z/1000)/298.15
    else:       # stratosphere
        T_a=226.65/298.15
    return TZERO*T_a


def q_sat(T,P):
    """
    Calculate saturation specific humidity (gr water vapor per gram moist air).
    inputs:
    T: temperature, in Kelvin
    P: pressure, in mb
    """
    R_v = 461 # Gas constant for moist air = 461 J/(kg*K)
    R_d = 287 # Gas constant 287 J K^-1 kg^-1
    TT = T-273.15 # Kelvin to Celsius
    # Saturation water vapor pressure (mb) from Emanuel 4.4.14 p 116-117:
    ew = 6.112*np.exp((17.67 * TT) / (TT + 243.5))
    # saturation mixing ratio (gr water vapor per gram dry air):
    rw = (R_d / R_v) * ew / (P - ew)
    # saturation specific humidity (gr water vapor per gram moist air):
    qw = rw / (1 + rw)
    return qw
```

2a) The parcel is lifted from the surface to z=3 km. Plot the MSE conservation (LHS vs RHS of equation 7.1) as a function of T, and find the parcel's final temperature as the one for which the LHS equals the RHS. Is the parcel saturated at that point?

[ ]:
```
# specify properties of the parcel at the surface:
T_surf=280.0
z_surf=0.0
RH_surf=0.7
q_surf=RH_surf*q_sat(T_surf,1000)
MSE_surf = cp_dry*T_surf+g*z_surf+L*q_surf

# height and pressure to which parcel is raised:
z=3000 # meters
# Assume isothermal atmosphere to calculate the atmospheric pressure at z:
T_const = 273.15
p = PZERO*np.exp(-g*z/R_dry/T_const)/100 # convert pressure to mb for q_sat

# plot MSE conservation allowing for the parcel to be
# either undersaturated or saturated:
T_plot=np.arange(240.0,280.0,1.0)
MSE_plot=T_plot*0.0
for i in range(0,len(T_plot)):
    MSE=XX
    MSE_plot[i]=MSE

MSE_surf_plot=XX # a constant, does not depend on Temprature
```

```
plt.figure(figsize=(12,6))
# plot in units of temperature by dividing by cp_dry:
plt.plot(T_plot,MSE_surf_plot/cp_dry,label="XX");
plt.plot(T_plot,MSE_plot/cp_dry,label="XX");
plt.legend(XX)
plt.ylabel(XX)
plt.xlabel("$T$")

# Given the solution for the temperature of the parcel,
# check if the parcel is expected to be saturated:
T_solution= 260 # get this value from the graph
qsat=q_sat(T_solution,p)
print("q_surf,qsat(z)=",q_surf,qsat)
#if XX
#    print("q_surf>qsat, parcel must be saturated at this new level")
#else:
#    print("q_surf<qsat, parcel is not saturated at this new level")
```

2b) Plot the parcel's profiles of temperature and relative humidity as it rises for z=0, 0.1, . . . , 10 km. Repeat for an initial RH of 50, 70, and 90 percent. Describe the results: At what heights are the LCL? LFC? LNB?

[ ]:
```
# Define the height coordinates, calculate background pressure
# and temperature profiles.
z = np.linspace(0,12000,1001) # z in m
p = np.zeros(np.shape(z))  # initialize P array
T_env = np.zeros(np.shape(z))  # initialize T array

# Assume isothermal atmosphere to calculate the atmospheric pressure at z:
T_const = 273.15
p = PZERO*np.exp(-g*z/R_dry/T_const)/100 # convert pressure to mb for q_sat

for i in range(len(z)):
    T_a = SimpleAtmosphere(z[i])
    T_env[i] = T_a

def MSE_conservation(T,P,z,MSE_s,q_s):
    """ This function is called by a root finder to calculate
    the temperature of a rising parcel. It returns the RHS minus the LHS and
    is equal to zero when MSE conservation is satisfied.
    Input arguments:
        temperature, pressure, height (m) at which conservation is calculated;
        MSE_s is the MSE to which we are trying to match;
        q_s is the specific humidity of the parcel before rising to height=z;
    """
    qsat=q_sat(T,P)
    # the root finder is looking for the value of T for which the following is zero:
    ans = cp_dry*T+L*min(q_s,qsat)+g*z-MSE_s
    return(ans)


def calc_T_and_RH_profiles_in_convection(p_s,T_s,z_s,RH_s):
    """ Use conservation of MSE to calculate the temperature and RH profiles of a
        adiabatically lifted moist parcel.
```

```
    """

    q_s=RH_s*q_sat(T_s,p_s)
    # initial MSE at surface, to be conserved by rising air parcel:
    MSE_s = np.nan # XX fix this line
    T_parcel = np.zeros(np.shape(z))
    RH_parcel = np.zeros(np.shape(z))

    T_parcel[0] = T_s
    RH_parcel[0] = RH_s
    for i in range(1,len(z)):
        sol = optimize.root(MSE_conservation, T_parcel[i-1],␣
 ↪args=(p[i],z[i],MSE_s,q_s))
        T_parcel[i] = float(sol.x[0])
        qsat=q_sat(T_parcel[i],p[i])
        q=min(q_s,qsat)
        RH_parcel[i] = np.nan # min(XX) fix this line

    return(RH_parcel,T_parcel)


# initiaize figure and two subplots for temperature and relative humidity:
plt.figure(figsize=(8,6),dpi=200)
ax1=plt.subplot(1,2,1)
ax2=plt.subplot(1,2,2)

# calculate and plot a convection profile:
p0 = p[0] # surface pressure
T0 = 273.15 + 25 #T[0] surface temperature
z0 = z[0] # surface height
RH = 0.7 # initial relative humidity
RHpath,T_parcel = calc_T_and_RH_profiles_in_convection(p0,T0,z0,RH)
ax1.plot(T_parcel,z/1000,label="T parcel")
# plot environment temperature profile:
ax1.plot(T_env,z/1000,color='k',linestyle='--',alpha=0.75,label="T environment")  #␣
 ↪Plot atmospheric background.
ax1.legend()
ax1.set_xlabel('T (K)')
ax1.set_ylabel('z (km)')
ax1.set_title('(a) Temperature')
ax1.grid()

# second subplot:
ax2.plot(100*RHpath,z/1000,label="RH")
ax2.legend()
ax2.set_xlabel('RH (%)')
ax2.set_title('(b) Relative Humidity')
ax2.grid();
#ax2.set_ylabel('z (km)')

XX Repeat for an initial RH of 50, 70, and 90 percent.
```

Describe the results for an initial RH of 70 percent:

XX

2c) Qualitatively, no equations: if a volume of rising air entrains the same volume of dry air from its surroundings when at 1 km as it rises, what do you expect the consequence to be for its motion, temperature, RH and LFC? Solution: XX.

2d) Optional extra credit: calculate the parcel profiles of temperature and relative humidity as it rises as above, assuming that the parcel entrains environmental air of a fraction 0.1 of its volume per km. Assume that the environmental air has a lapse rate of 6.5 deg/km and that its RH is 70%. Hint: raise the parcel adiabatically 100 m, entrain air from the environment and calculate the resulting averaged parcel temperature and moisture. adjust the moisture to be no more than the saturation moisture, and repeat.

[ ]:
```
# code here. XX
```

3) Cloud feedbacks in a 2-level energy balance model: Global mean surface temperature is predicted to increase by 2–4 ∘C due to doubling of CO2

3a) Calculate the steady state temperatures for both CO_2=280 without cloud feedbacks, use the solution as the initial conditions as well as the reference temperature in the cloud feedback terms. Now run a warming scenario CO_2 increasing exponentially on a time scale of 150 years until doubling, and continuing at double CO_2. Repeat this run without cloud feedbacks and then with cloud feedbacks, using $\Delta_{LW} = 0.001$ and $\Delta_{SW} = -0.01$. Plot the time series of CO_2, T, T_a, emissivity and albedo for the runs with and without cloud feedbacks. Discuss the effects of cloud feedbacks on climate sensitivity (equilibrium warming due to $\times 2CO_2$) in this case.

[ ]:
```
#########################################################################
def rhs(time,T,C_ocn,C_atm,CO2_initial,CO2_fixed,Delta_SW,Delta_LW):
    #########################################################################
    # right hand side of the equations for T_ocn and T_a:
    T_ocn=T[0]
    T_a=T[1]
    # calculate the RHS of both equations (rhs is an array of two elements):
    rhs=np.array([ (S*(1-alpha(T_ocn,Delta_SW))
                    ␣
  ↪+epsilon(T_a,Delta_LW,CO2_initial,CO2_fixed,time)*sigma*T_a**4-sigma*T_ocn**4)/C_ocn
                   ,(XX)/C_atm])
    return rhs


#########################################################################
def alpha(T,Delta_SW):
    # albedo as function of temperature representing low cloud effects
    #########################################################################
    alpha0=0.3
    alph=alpha0*(1+Delta_SW*(T-T_ref))
    return alph


#########################################################################
def epsilon(T_a,Delta_LW,CO2_initial,CO2_fixed,time):
```

```
    # atmospheric emissivity as function of temperature, representing high cloud␣
 ↪effects
    ##########################################################################
    epsilon0=0.75+0.05*np.log2(CO2(CO2_initial,CO2_fixed,time)/280)
    eps=XX
    return np.minimum(1.0,eps)


##############################################################################
def CO2(CO2_initial,CO2_fixed,time):
    # atmospheric CO2 as function of time;
    # if CO2_fixed=True, CO2_initial is used for all times.
    ##########################################################################
    if CO2_fixed:
        CO2=CO2_initial
    else:
        CO2=CO2_initial*np.exp(XX)
    return np.minimum(CO2,560)
```

[ ]:
```
##############################################################################
## Main program starts here.
##############################################################################

# set parameters:
year=365*24*3600;
time_max=200*year;
Cp_w=4000
rho_w=1000
C_ocn=Cp_w*rho_w*50
C_atm=C_ocn/5
sigma=5.67e-8
S=1361/4
CO2_initial=280
T_ref=281.41
T_a_ref=236.64


# run the energy balance model to steady state for CO2=280
# to get initial conditions for warming scenario:
# ------------------------------------------------------------
T_init=[280,250] # initial conditions
teval=np.arange(0,time_max,time_max/100)
tspan=(teval[0],teval[-1])
# without feedbacks:
Delta_SW=0.0  # try -0.02 to 0.1
Delta_LW=0.0  # try +/- 0.02
CO2_fixed=True
sol = solve_ivp(fun=lambda time,T:␣
 ↪rhs(time,T,C_ocn,C_atm,CO2_initial,CO2_fixed,Delta_SW,Delta_LW) \
                ,vectorized=False,y0=T_init,t_span=tspan,t_eval=teval,rtol=1.0e-6)
time_no_feedback=sol.t
T_280=sol.y[:,-1]
T_ref=T_280[0]
```

```
T_a_ref=T_280[1]
print("steady temperature (surface/atmospher) at CO2=280:",T_280)
```

[ ]:
```
# run the energy balance model in time-dependent warming scenario:
# ----------------------------------------------------------------
T_init=[XX,XX] # initial conditions
teval=np.arange(0,time_max,time_max/100)
tspan=(teval[0],teval[-1])
# without feedbacks:
Delta_SW=XX
Delta_LW=XX
CO2_fixed=False
sol = solve_ivp(fun=lambda time,T:␣
 ↪rhs(time,T,C_ocn,C_atm,CO2_initial,CO2_fixed,Delta_SW,Delta_LW) \
               ,vectorized=False,y0=T_init,t_span=tspan,t_eval=teval,rtol=1.0e-6)
time_no_feedback=sol.t
T_no_feedback=sol.y
# with feedbacks:
Delta_SW=XX
Delta_LW=XX
CO2_fixed=False
sol = solve_ivp(fun=lambda time,T:␣
 ↪rhs(time,T,C_ocn,C_atm,CO2_initial,CO2_fixed,Delta_SW,Delta_LW) \
               ,vectorized=False,y0=T_init,t_span=tspan,t_eval=teval,rtol=1.0e-6)
time=sol.t
T=sol.y


# save model output to be plotted:
# --------------------------------
T_plot=np.zeros(len(T[0,:]))
T_a_plot=np.zeros(len(T[0,:]))
T_plot_no_feedback=np.zeros(len(T[0,:]))
T_a_plot_no_feedback=np.zeros(len(T[0,:]))
alpha_plot=np.zeros(len(T[0,:]))
epsilon_plot=np.zeros(len(T[0,:]))
CO2_plot=np.zeros(len(T[0,:]))
i=0;
for t in T_plot:
    T_plot[i]=T[0,i];
    T_a_plot[i]=T[1,i];
    T_plot_no_feedback[i]=T_no_feedback[0,i];
    T_a_plot_no_feedback[i]=T_no_feedback[1,i];
    alpha_plot[i]=alpha(XX)
    epsilon_plot[i]=epsilon(T[1,i],Delta_LW,CO2_initial,CO2_fixed,time[i])
    CO2_plot[i]=CO2(XX)
    i=i+1;

N=len(T_plot)
```

[ ]:
```
#############################################################################
## plots:
#############################################################################
```

```
fig1=plt.figure(1,figsize=(8,4),dpi=150)

plt.subplot(2,2,1)
plt.plot(time_no_feedback/year,CO2_plot,'b-')
plt.xlabel(XX);
plt.ylabel('CO$_2$ (ppm)');
plt.grid(lw=0.25)
axes=plt.gca()
plt.text(0.05, 0.5, "(a)", transform=axes.transAxes, fontsize=14,
        verticalalignment='top')


plt.subplot(2,2,2)
h1=plt.plot(time/year,alpha_plot,'b-',markersize=1,label="$\\alpha$")
plt.grid(lw=0.25)
axis1=plt.gca()
axis2 =axis1.twinx()
h2=axis2.plot(time/year,epsilon_plot,'r-',markersize=1,label="$\\epsilon$")
plt.xlabel(XX);
plt.ylabel(XX);
plt.title(XX);
# added these three lines
h = h1+h2
legend_labels = [l.get_label() for l in h]
axis1.legend(h, legend_labels, loc="center right")
axis1.tick_params(axis='y', labelcolor="b")
axis2.tick_params(axis='y', labelcolor="r")
axes=plt.gca()
plt.text(0.05, 0.5, "(b)", transform=axes.transAxes, fontsize=14,
        verticalalignment='top')


plt.subplot(2,2,3)
plt.plot(time_no_feedback/year,T_plot_no_feedback,'b--',markersize=1,label="$T$, no␣
 ↪feedback")
plt.plot(time/year,T_plot,'b-',markersize=1,label="T, with feedback")
plt.xlabel(XX);
plt.ylabel(XX);
plt.ylim([286,291.5])
plt.title(XX);
plt.legend()
plt.grid(lw=0.25)
axes=plt.gca()
plt.text(0.05, 0.5, "(c)", transform=axes.transAxes, fontsize=14,
        verticalalignment='top')

plt.subplot(2,2,4)
plt.plot(time_no_feedback/year,T_a_plot_no_feedback,'b--',markersize=1,label="$T_a$,␣
 ↪no feedback")
plt.plot(time/year,T_a_plot,'b-',markersize=1,label="$T_a$, with feedback")
plt.xlabel(XX);
plt.ylabel(XX);
```

```
plt.ylim([240,245])
#plt.title('$T_a$');
plt.legend()
plt.grid(lw=0.25)
axes=plt.gca()
plt.text(0.05, 0.5, "(d)", transform=axes.transAxes, fontsize=14,
        verticalalignment='top')


plt.tight_layout()
plt.show()

#fig1.savefig("Output/clouds-energy-balance-with-cloud-feedbacks.pdf")
```

3b) Estimate the SW cloud feedback parameter Δ_{SW} needed to increase the predicted warming due to the CO_2 doubling by 2C. To do this, tune Δ_{SW} until the difference in temperature between 560ppm and 280ppm is now 2C higher than the difference in temperature with Δ_{SW} set to zero. Next, estimate the percent change in albedo between 560ppm and 280ppm with nonzero Δ_{SW}. Discuss the sign and is this what you would expect to cause warming?

[ ]:
```
# First, no cloud feedbacks:
# ------------------------------
CO2_fixed=True
CO2_initial=280.0
Delta_SW=0.0
Delta_LW=0.0
sol = solve_ivp(fun=lambda time,T:␣
 ↪rhs(time,T,C_ocn,C_atm,CO2_initial,CO2_fixed,Delta_SW,Delta_LW) \
                ,vectorized=False,y0=T_init,t_span=tspan,t_eval=teval,rtol=1.0e-6)
T_steady=sol.y[:,-1]
print("CO2=",CO2_initial,", Delta_SW=",Delta_SW,", Delta_LW=",Delta_LW)
print("The steady state temperature [T, T_a]=",T_steady)

CO2_initial=560.0
sol = solve_ivp(fun=lambda time,T:␣
 ↪rhs(time,T,C_ocn,C_atm,CO2_initial,CO2_fixed,Delta_SW,Delta_LW) \
                ,vectorized=False,y0=T_init,t_span=tspan,t_eval=teval,rtol=1.0e-6)
T_steady=sol.y[:,-1]
print("CO2=",CO2_initial,", Delta_SW=",Delta_SW,", Delta_LW=",Delta_LW)
print("The steady state temperature [T, T_a]=",T_steady)

# Next, with SW cloud feedbacks:
# ------------------------------
# repeat the above with SW cloud feedback
# XX
print("CO2=",CO2_initial,", Delta_SW=",Delta_SW,", Delta_LW=",Delta_LW)
print("The steady state temperature [T, T_a]=",T_steady)
```

3c) Estimate the LW cloud feedback parameter Δ_{LW} needed to decrease the predicted warming due to the CO_2 doubling by 2C. To do this, tune Δ_{LW} until the difference in temperature between 560ppm and 280ppm is now 2C lower than the difference in temperature with Δ_{LW} set to zero. Next, estimate the percent change in emissivity between 560ppm and 280ppm with nonzero Δ_{LW}. Discuss the sign and is this what you would expect to cause cooling?

[ ]:
```
# First, no cloud feedbacks:
# --------------------------
CO2_initial=280.0
Delta_SW=0.0
Delta_LW=0.0
sol = solve_ivp(fun=lambda time,T:␣
 ↪rhs(time,T,C_ocn,C_atm,CO2_initial,CO2_fixed,Delta_SW,Delta_LW) \
               ,vectorized=False,y0=T_init,t_span=tspan,t_eval=teval,rtol=1.0e-6)
T_steady=sol.y[:,-1]
print("CO2=",CO2_initial,", Delta_SW=",Delta_SW,", Delta_LW=",Delta_LW)
print("The steady state temperature [T, T_a]=",T_steady)

CO2_initial=560.0
sol = solve_ivp(fun=lambda time,T:␣
 ↪rhs(time,T,C_ocn,C_atm,CO2_initial,CO2_fixed,Delta_SW,Delta_LW) \
               ,vectorized=False,y0=T_init,t_span=tspan,t_eval=teval,rtol=1.0e-6)
T_steady=sol.y[:,-1]
print("CO2=",CO2_initial,", Delta_SW=",Delta_SW,", Delta_LW=",Delta_LW)
print("The steady state temperature [T, T_a]=",T_steady)

# Next, with LW cloud feedbacks:
# ------------------------------
# repeat the above with LW cloud feedback
# XX
print("CO2=",CO2_initial,", Delta_SW=",Delta_SW,", Delta_LW=",Delta_LW)
print("The steady state temperature [T, T_a]=",T_steady)
```

[ ]:
```

```